Distributed URL Shortener

Alvyn Kwon, Jinhee Lee, Krushay Bhavsar, Wesley Jeong, Yasha Rachakonda

(1) Motivation/Objective

Due to the rapid growth of the Internet and social media platforms, there has been an increasing reliance on hyperlinks for sharing information. However, many web addresses are long and complex, making them difficult to distribute efficiently in messaging platforms, printed materials, QR codes and character limited environments. To address this, many URL shortening services like Tinyurl and Bitly have been made available; they provide easy to share links that redirect users to the original URL. However, the underlying service infrastructure must also be capable of handling high volumes of traffic. If the server crashes or gets overloaded, every short link breaks.

This problem tackles the problem of a URL shortening service that avoids the limitations of a single server architecture. We intend to tackle this problem by offering a distributed URL shortening system composed of multiple backend service instances and a front end gateway (load balancer) that routes requests to valid nodes. We will provide a REST API that accepts a long URL and generates a short code, redirecting users from the short code to the original URL. A shared database will be used so all backend nodes can access consistent mapping. In addition, we plan on implementing health checks so the gateway only routes requests to the active backend nodes. This ensures that if one of the backend nodes fails, the system continues to operate without interruptions.

We believe this project is useful because it provides a compact and realistic application of Internet systems principals. Although the application domain is simple, it mirrors real-world service architectures used in modern web infrastructure. By implementing this distributed URL shortener, we believe we will gain practical insight into how modern Internet services are designed to remain scalable and efficient.

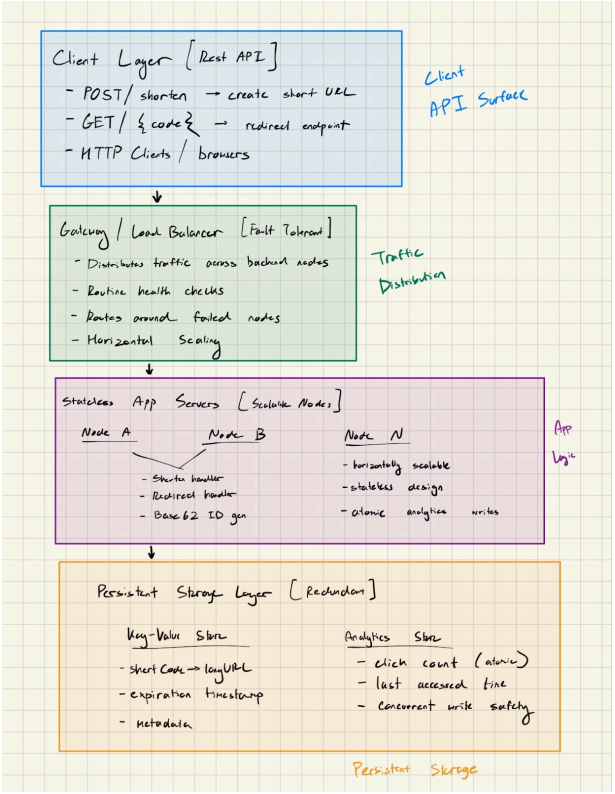
(2) Related Work

URL shortening systems have been studied extensively throughout the advent of the internet, as they often serve as prime examples of technology that explore scalability, security, and performance. In their paper “Optimized URL Shortener Using Cloud Platform” [1], Wani et al. describe a fully cloud-based architecture that makes use of distributed infrastructure to drastically improve the reliability and scalability of the platform. The paper goes in depth on how deploying load balancers and cloud environments can improve how large volumes of requests can be handled. Even though this would improve the availability of the system, it would still rely

a lot on managed web services and does not explore the pros and cons of a centralized versus a distributed ID generation approach.

In their paper, “URL Shortener for Web Consumption” [2], Gochhait et al. highlight the importance of generating short codes and protecting against malicious URL injections. The paper focuses more on encryption-based algorithms and various validation techniques to prevent bad actors from misusing the service. Even though their approach to URL shorteners would significantly improve the security of the system, it does not extensively cover the issue of performance under high load. Node failures are also a common issue in real-world distributed systems and are worth exploring. A deeper analysis of URL shortening algorithms is given in the paper “Comparative Analysis of URL Shortening Algorithms” [3]. The paper introduces several valuable insights on short code generation techniques like hashing and Base62 encoding. It compares them based on properties like collision probability, lookup efficiently, and storage, all of which are relevant considerations for scalable real-world systems. For our project, we hope to build upon the topics explored in each of these papers and integrate efficient short code generation, fault tolerant persistence, and scalability into one robust URL shortener system.

(3) Proposed Work:



Our URL shortener is meant to be more than just a simple URL mapping. Unlike traditional shorteners, we plan to emphasize availability and scalability by focusing on making it a distributed and fault-tolerant URL shortening service. Some of the services that our system will provide is the initial URL shortening REST API as well as basic analytics such as last accessed, click count, and link expiration details. Normal URL shorteners are single-server implementations, whereas ours will be deployed across multiple nodes behind a node balancer as a means to increase fault tolerance and allow for horizontal scaling.

To go more in depth, our system will have four key components: the client layer, load balancer, backend nodes, and storage layer. The first layer, the client layer, will serve as a redirect endpoint for resolving short URLs. This layer will involve exposing the REST API endpoints to create the shortened links. Then all the requests will pass through the next layer, the load balancer, which helps to distribute traffic across the backend nodes.

Additionally it will perform routine health checks and will route around the nodes that fail. The third layer is the backend application layer which has stateless servers that handle the requests for shortening and redirecting the URLs. To emphasize distribution, the nodes will be scalable and communicate with the last component, the storage layer. The storage layer will have a database or a key-value store to map the shortCode to longURL. The database will also maintain data such as expiration time and other analytics as mentioned from before.

The project will also address technical challenges ranging from short code generation to concurrency. We plan to evaluate Base62 encoding of unique IDs, where we will compare centralized and distributed ID strategies. As mentioned before we will also focus on implementing fault tolerant mechanisms such as backend node failure handling and database reliability through redundancy. We will also explore scalability, specifically backend node scalability. Finally we will ensure safe concurrent writes and atomic updates for analytics.

The key novelty for the project is treating the URL shortener as a distributed service, where we will emphasize consistency versus availability trade offs, performance behavior under load, and overall system behavior when nodes fail. Instead of just focusing on functionality, we will compare performance between single-node and multi-node deployments as a means to the improvements in latency, throughput, and more.

Finally, the main evaluation of success will be based on a few criteria. For one, functional correctness is imperative as we will ensure accurate redirects and proper expirational handling. Additionally fault tolerance will be evaluated based on if the system continues to function despite node failures. We will also quantify performance metrics such as latency and throughput metrics under load. Scalability will also be assessed by how much it improves as we add more backend nodes. All together, the project will help to demonstrate not only the effectiveness of a distributed URL shortener but also whether the system will meet its goals of availability and reliability.

(4) Plan of Action

We will focus on utilizing open-source software. For instance, for the backend services we will implement modern web frameworks like FastAPI or Node. For our storage we will utilize PostgreSQL or Redis. Nginx is also necessary as a load balancer in order to distribute traffic across nodes. Docker will then be used to containerize services and replicate the distributed environment.

When it comes to testing and development, it will be done on our own devices. If necessary we can utilize AWS or GCP to deploy our systems on the cloud. To measure latency or throughput we can use existing services like Locust or Apache JMeter.

Dates	Plan
02/27/26	Proposal Due
3/04/26	Proposal Meeting; finalize architecture/ideas
3/11/26	Detailed API specification and design
3/18/26	Core single-node service (redirecting endpoints)
3/25/26	Integrate persistent database and expiration logic
4/01/26	Analytics feature and concurrency
4/08/26	Multi-node backend w/ load balancer
4/15/26	Test for fault tolerance and scalability; collect metrics
4/20/26	Presentation schedule
4/21/26	Final demo

(5) Evaluation and Testing Method

For evaluation, we will mainly look at four main areas which are functional correctness, performance, scalability, and fault tolerance. Furthermore, the testing strategy brings together unit tests, integration tests, and controlled experiments under load. This will allow us to systematically validate whether the system maintains high availability and reliability.

For functional testing, we can implement unit tests for components such as short code generation, URL validation, database persistence, and analytics updates. In fact, integration tests will validate the full REST workflow, creating a short URL and correctly redirecting to the original link. Additionally, edge cases will be tested as well including invalid URLs, duplicate requests, and expired links.

For performance and scalability, we can perform load testing on both a single-node baseline and our multi-node distributed system. As a matter of fact, we simulate “shorten-heavy” and “read-heavy” workloads using a concurrent request generator. Moreover, we measure average and tail latency, throughput, and error rate. In order to evaluate scalability, we can increase the number of backend nodes and observe whether throughput increases and latency remains stable as load grows.

To evaluate fault tolerance and availability, we can introduce controlled failures during load testing by disabling backend nodes or emulating network disruptions. Then, we measure how fast the load balancer identifies failures and keeps service availability. In fact, maintaining uninterrupted service with minimal decline in performance and zero data loss would define success.

For consistency and persistence, we test concurrent access to ensure consistent URL mappings and atomic analytics updates across all nodes. In fact, system restarts will verify that persistent storage maintains data integrity and that services recover without losing links.

Lastly, for baseline comparison, we compare our distributed implementation to a single-server version using identical workloads. Additionally, metrics like latency, throughput, and availability convey whether distribution improves scalability and resilience.

(6) Bibliography

[1] V. Wani, A. Paul, and K. Kumar, “Optimised URL Shortener using Cloud Platform,” International Journal of Computer Science and Mobile Computing, vol. 10, no. 5, pp. 95–101, May 2021, doi: https://doi.org/10.47760/ijcsmc.2021.v10i05.010.

[2] Saikat Gochhait et al., “URL shortener for web consumption: an extensive and impressive security algorithm,” Indonesian journal of electrical engineering and computer science, vol. 35, no. 1, pp. 284–284, Jul. 2024, doi: https://doi.org/10.11591/ijeecs.v35.i1.pp284-291.

[3] І.В. Кириченко and Є.О. Шамрай, “Comparative analysis of URL shortening algorithms: Performance considerations in the application domain of web search and information retrieval technology,” Біоніка інтелекту, vol. 1, no. 98, pp. 98–105, 2022, doi: https://doi.org/10.30837/bi.2022.1(98).13.`